MA388 Sabermetrics: Lesson 4

Introduction to R Graphics - Part I

Markov Chains

Markov chains model the probabilities of moving between different states. If our goal is to model a baseball game, how could we employ Markov chains? Which states are relevant?

What is the memoryless property of Markov chains? Does this allow for "momentum?"

```
library(tidyverse)
library(knitr)

# Load the data.
site = "https://raw.githubusercontent.com/maxtoki/baseball_R/"
fields <- read_csv(file = paste(site, "master/data/fields.csv", sep =""))
retro2011 <- read_csv(file = paste(site, "master/data/all2011.csv", sep = ""),
                      col_names = pull(fields, Header),
                      na = character())
colnames(retro2011) <- tolower(colnames(retro2011))
```

Let's add states, etc., similarly to how we've done it in previous lessons. One important difference is that we don't care if/where baserunners were left on base

when the third out was made. The third out is, however, an important state for us to capture.

```r
# Add states, etc.
retro2011 <- retro2011 |>
  mutate(runs_before = away_score_ct + home_score_ct,
         half_inning = paste(game_id, inn_ct, bat_home_id),
         runs_scored =
           (bat_dest_id > 3) + (run1_dest_id > 3) +
           (run2_dest_id > 3) + (run3_dest_id > 3))

half_innings <- retro2011 |>
  group_by(half_inning) |>
  summarize(outs_inning = sum(event_outs_ct),
            runs_inning = sum(runs_scored),
            runs_start = first(runs_before),
            max_runs = runs_inning + runs_start)

retro2011_complete <- retro2011 |>
  inner_join(half_innings, by = "half_inning") |>
  mutate(runs_roi = max_runs - runs_before) |>
  mutate(bases =
           paste(ifelse(base1_run_id > '',1,0),
                 ifelse(base2_run_id > '',1,0),
                 ifelse(base3_run_id > '',1,0), sep = ""),
         state = paste(bases, outs_ct)) |>
  mutate(is_runner1 =
           as.numeric(run1_dest_id==1 | bat_dest_id == 1),
         is_runner2 =
           as.numeric(run1_dest_id == 2 | run2_dest_id == 2 |
                        bat_dest_id == 2),
         is_runner3 =
           as.numeric(run1_dest_id == 3 | run2_dest_id == 3 |
                        run3_dest_id == 3 | bat_dest_id == 3),
         new_outs = outs_ct + event_outs_ct,
         new_bases = paste(is_runner1, is_runner2, is_runner3, sep = ""),
         new_state = paste(new_bases, new_outs)) |>
  filter((state != new_state) | (runs_scored > 0)) |>
  filter(outs_inning == 3, bat_event_fl == TRUE) |>
  mutate(new_state = str_replace(new_state, "[0-1]{3} 3", "3"))
```

Let's look at our resulting data frame:

```
retro2011_complete |>
  select(game_id, bat_id, state, new_state) |>
  head(10) |>
  kable()
```

game_id	bat_id	state	new_state
ANA201104080	davir003	000 0	000 1
ANA201104080	nix-j001	000 1	000 2
ANA201104080	bautj002	000 2	3
ANA201104080	iztum001	000 0	100 0
ANA201104080	kendh001	100 0	100 1
ANA201104080	abreb001	000 2	100 2
ANA201104080	huntt001	100 2	001 2
ANA201104080	wellv001	001 2	3
ANA201104080	linda001	000 0	000 1
ANA201104080	hilla001	000 1	000 2

We're now ready to compute a matrix of transition probabilities. There are 24 potential starting states and 25 potential ending states (24 + 3 outs). How can we compute these state-to-state probabilities?

```
T_matrix <- retro2011_complete  |>
  select(state, new_state) |>
  table()
dim(T_matrix)
```

```
[1] 24 25
```

```
P_matrix <- prop.table(T_matrix, 1)
dim(P_matrix)
```

```
[1] 24 25
```

Once you get to three outs, there's nowhere left to go – you will stay in three outs with probability 1.

```r
P_matrix <- P_matrix |>
  rbind("3" = c(rep(0, 24), 1))
dim(P_matrix)
```

```
[1] 25 25
```

```r
# P_matrix |> round(3)
```

How can we check the validity of this transition matrix? Pick a state – what should the sum of all transition probabilities from that state to any other be?

```
P_matrix |>
  apply(MARGIN = 1, FUN = sum)
```

```
000 0 000 1 000 2 001 0 001 1 001 2 010 0 010 1 010 2 011 0 011 1 011 2 100 0
    1     1     1     1     1     1     1     1     1     1     1     1     1
100 1 100 2 101 0 101 1 101 2 110 0 110 1 110 2 111 0 111 1 111 2     3
    1     1     1     1     1     1     1     1     1     1     1     1
```

Let's examine the nobody-on, nobody-out state. What are the most likely resulting states?

```
P_matrix |>
  as_tibble(rownames = "state")  |>
  filter(state == "000 0") |>
  pivot_longer(
    cols = -state,
    names_to = "new_state",
    values_to = "Prob"
  ) |>
  filter(Prob > 0) |>
  kable()
```

state	new_state	Prob
000 0	000 0	0.0266392
000 0	000 1	0.6788661
000 0	001 0	0.0057448
000 0	010 0	0.0498403
000 0	100 0	0.2389096

What about with a runner on 2nd base and two outs?

```
P_matrix |>
  as_tibble(rownames = "state") |>
```

```
  filter(state == "010 2") |>
  pivot_longer(
    cols = -state,
    names_to = "new_state",
    values_to = "Prob"
  ) |>
  filter(Prob > 0) |>
  kable()
```

state	new_state	Prob
010 2	000 2	0.0196963
010 2	001 2	0.0061551
010 2	010 2	0.0545753
010 2	011 2	0.0002736
010 2	100 2	0.0815210
010 2	101 2	0.0339215
010 2	110 2	0.1615374
010 2	3	0.6423198

So far, so good… Now we need to actually simulate the Markov Chain. This involves being in a state and transitioning into possible other states in a probabilistic manner…a large number of times.

The key equation for keeping score is:

$$runs = \left(N_{runners}^{(before)} + Outs^{(before)} + 1\right) - \left(N_{runners}^{(after)} + Outs^{(after)}\right)$$

```
# This function essentially "adds" the state: "011 1" becomes 3.
num_havent_scored <- function(s) {
  s |>
    str_split("") |>
    pluck(1) |>
    as.numeric() |>
    sum(na.rm = TRUE)
}

# "Add" the state (runners + outs) for every state.
runners_out <- T_matrix |>
  row.names() |>
  set_names() |>
  map_int(num_havent_scored)

# This operation computes the runs equation above for every state-to-state
```

```r
# transition, including the impossible transitions that we can safely ignore.
R_runs <- outer(
  runners_out + 1,
  runners_out,
  FUN = "-"
) |>
  cbind("3" = rep(0, 24))
```

Before we go any further, what are the key tasks for simulating a half-inning of baseball?

Here we go - let's simulate a half-inning of baseball!

```r
# This function returns the number of runs scored in a simulated half-inning.
simulate_half_inning <- function(P, R, start = 1) {
  s <- start
  path <- NULL
  runs <- 0
  while (s < 25) {
    s_new <- sample(1:25, size = 1, prob = P[s, ])
    path <- c(path, s_new)
    runs <- runs + R[s, s_new]
    s <- s_new
  }
  runs
}
```

Let's simulate 10,000 half-innings.

```r
set.seed(388)
simulated_runs <- 1:10000 |>
  map_int(~simulate_half_inning(T_matrix, R_runs))
```

What was the observed distribution of half-inning run totals?

```r
table(simulated_runs)
```

```
simulated_runs
   0    1    2    3    4    5    6    7    8    9
7441 1398  655  289  128   57   22    5    4    1
```

How often do teams score at least five runs in an inning?

```
sum(simulated_runs >= 5) / 10000
```

```
[1] 0.0089
```

What is the average number of runs scored per half-inning?

```
mean(simulated_runs)
```

```
[1] 0.458
```

What have we left out? Name some non-batting plays that have the potential to change the state and potentially score runs.